



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«МИРЭА – Российский технологический университет»
РТУ МИРЭА**

Институт искусственного интеллекта

Кафедра высшей математики

Отчёт по лабораторной работе № 2

Вариант 3

по дисциплине «Численные методы»

Выполнил студент группы КМБО-07- 22

Баттур Ц.

Проверил

Алексеев А.А.

Москва 2025

Задание

Разработать программу реализующий указанный метод решения для систем линейных алгебраических уравнений

$$AX = B$$

с матрицей A произвольного порядка n и решить с её помощью заданную систему. Для полученного решения X прямой подстановкой вычислить невязку

$$R = \|AX - B\|$$

по норме указанной в варианте. В разрабатываемой программе предусмотреть вывод промежуточных результатов (матриц L и U для LU -разложения, матрицы L или U для метода Холецкого и прогоночных коэффициенты α_i и β_i для метода прогонки) для визуализации и контроля алгоритмов. Для вариантов с LU -разложением и методом Холецкого вычислить также детерминант матрицы системы $|A|$.

Метод прогонки. Невязка по норме $\|\cdot\|_\infty$.

$$\begin{pmatrix} 2 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 7 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 19 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 6 & 37 & 12 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 19 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 10 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix} \vec{x} = \begin{pmatrix} 9 \\ 27 \\ 9 \\ -135 \\ -59 \\ -20 \\ 8 \\ -2 \end{pmatrix}$$

Программа, реализующая алгоритм метода прогонки и находящая α , β и x

```
import numpy as np

def tdmsolver(a, b, c, d):
    n = len(b)
    alpha = np.zeros(n)
    beta = np.zeros(n)

    alpha[0] = -c[0] / b[0]
    beta[0] = d[0] / b[0]

    print("Initialization")
    print(f"alpha[0] = -{c[0]:.5f} / {b[0]:.5f} = {alpha[0]:.5f}")
    print(f"beta[0] = {d[0]:.5f} / {b[0]:.5f} = {beta[0]:.5f}")

    for i in range(1, n):
        denom = b[i] + a[i] * alpha[i-1]
        if i < n - 1:
            alpha[i] = -c[i] / denom
        else:
            alpha[i] = 0

        beta[i] = (d[i] - a[i] * beta[i - 1]) / denom
        print(f"знаменатель = b[{i}] + a[{i}]*alpha[{i-1}] = {b[i]} + {a[i]} * {alpha[i-1]} = {denom}")
        if i < n - 1:
            print(f"  alpha[{i}] = -c[{i}] / denom = -{c[i]} / {denom} = {alpha[i]}")
        else:
            print(f"  альфа[{i}] не используется (установлен в 0)")
            print(f"  beta[{i}] = (d[{i}] - a[{i}]*beta[{i-1}]) / denom = ({d[i]} - {a[i]}*{beta[i-1]}) / {denom} = {beta[i]}")

    x = np.zeros(n)
    x[-1] = beta[-1]
    print(f"x[{n-1}] = b[{n-1}] = {x[-1]}")
    for i in range(n - 2, -1, -1):
        x[i] = alpha[i] * x[i+1] + beta[i]
        print(f"x[{i}] = alpha[{i}] * x[{i+1}] + beta[{i}] = {alpha[i]} * {x[i+1]} + {beta[i]} = {x[i]}")

    return x, alpha, beta
```

Листинг 1. Программа, находящая α , β и x системы методом прогонки.

Данная программа получает на вход матрицу A , и из этой матрицы мы получаем α , β и x . После нахождения x мы проверяем это, умножая A и x .

$$A = \begin{pmatrix} 2 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 2 & 7 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 6 & 19 & 3 & 0 & 0 & 0 & 0 \\ 0 & 0 & 6 & 37 & 12 & 0 & 0 & 0 \\ 0 & 0 & 0 & 3 & 19 & 6 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 10 & 3 & 0 \\ 0 & 0 & 0 & 0 & 0 & 3 & 7 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}, \vec{b} = \begin{pmatrix} 9 \\ 27 \\ 9 \\ -135 \\ -59 \\ -20 \\ 8 \\ -2 \end{pmatrix}$$

Шаги решения представлены в Листинге 2:

```
A =
[[ 2.  1.  0.  0.  0.  0.  0.  0.]
 [ 2.  7.  1.  0.  0.  0.  0.  0.]
 [ 0.  6. 19.  3.  0.  0.  0.  0.]
 [ 0.  0.  6. 37. 12.  0.  0.  0.]
 [ 0.  0.  0.  3. 19.  6.  0.  0.]
 [ 0.  0.  0.  0.  3. 10.  3.  0.]
 [ 0.  0.  0.  0.  0.  3.  7.  0.]
 [ 0.  0.  0.  0.  0.  0.  0.  1.]]
Initialization
alpha[0] = -1.00000 / 2.00000 = -0.50000
beta[0]   = 9.00000 / 2.00000 = 4.50000
знаменатель = b[1] + a[1]*alpha[0] = 7.0 + 2.0 * -0.5 = 6.0
  alpha[1] = -c[1] / denom = -1.0 / 6.0 = -0.16666666666666666
  beta[1] = (d[1] - a[1]*beta[0]) / denom = (27.0 - 2.0*4.5) / 6.0 = 3.0
знаменатель = b[2] + a[2]*alpha[1] = 19.0 + 6.0 * -0.16666666666666666 = 18.0
  alpha[2] = -c[2] / denom = -3.0 / 18.0 = -0.16666666666666666
  beta[2] = (d[2] - a[2]*beta[1]) / denom = (9.0 - 6.0*3.0) / 18.0 = -0.5
знаменатель = b[3] + a[3]*alpha[2] = 37.0 + 6.0 * -0.16666666666666666 = 36.0
  alpha[3] = -c[3] / denom = -12.0 / 36.0 = -0.3333333333333333
  beta[3] = (d[3] - a[3]*beta[2]) / denom = (-135.0 - 6.0*-0.5) / 36.0 = -
3.6666666666666665
знаменатель = b[4] + a[4]*alpha[3] = 19.0 + 3.0 * -0.3333333333333333 = 18.0
  alpha[4] = -c[4] / denom = -6.0 / 18.0 = -0.3333333333333333
  beta[4] = (d[4] - a[4]*beta[3]) / denom = (-59.0 - 3.0*-3.6666666666666665) /
18.0 = -2.6666666666666665
знаменатель = b[5] + a[5]*alpha[4] = 10.0 + 3.0 * -0.3333333333333333 = 9.0
  alpha[5] = -c[5] / denom = -3.0 / 9.0 = -0.3333333333333333
  beta[5] = (d[5] - a[5]*beta[4]) / denom = (-20.0 - 3.0*-2.6666666666666665) / 9.0
= -1.3333333333333333
знаменатель = b[6] + a[6]*alpha[5] = 7.0 + 3.0 * -0.3333333333333333 = 6.0
  alpha[6] = -c[6] / denom = -0.0 / 6.0 = -0.0
  beta[6] = (d[6] - a[6]*beta[5]) / denom = (8.0 - 3.0*-1.3333333333333333) / 6.0 =
2.0
знаменатель = b[7] + a[7]*alpha[6] = 1.0 + 0.0 * -0.0 = 1.0
  альфа[7] не используется (установлен в 0)
  beta[7] = (d[7] - a[7]*beta[6]) / denom = (-2.0 - 0.0*2.0) / 1.0 = -2.0
x[7] = b[7] = -2.0
x[6] = alpha[6] * x[7] + beta[6] = -0.0 * -2.0 + 2.0 = 2.0
x[5] = alpha[5] * x[6] + beta[5] = -0.3333333333333333 * 2.0 + -1.3333333333333333
= -2.0
x[4] = alpha[4] * x[5] + beta[4] = -0.3333333333333333 * -2.0 + -2.6666666666666665
= -2.0
x[3] = alpha[3] * x[4] + beta[3] = -0.3333333333333333 * -2.0 + -3.6666666666666665
= -3.0
```

```

x[2] = alpha[2] * x[3] + beta[2] = -0.16666666666666666 * -3.0 + -0.5 = 0.0
x[1] = alpha[1] * x[2] + beta[1] = -0.16666666666666666 * 0.0 + 3.0 = 3.0
x[0] = alpha[0] * x[1] + beta[0] = -0.5 * 3.0 + 4.5 = 3.0
alpha =
[-0.50000 -0.16667 -0.16667 -0.33333 -0.33333 -0.33333 -0.00000 0.00000]
beta =
[4.50000 3.00000 -0.50000 -3.66667 -2.66667 -1.33333 2.00000 -2.00000]
x =
[3.00000 3.00000 0.00000 -3.00000 -2.00000 -2.00000 2.00000 -2.00000]

```

Листинг 2: Пошаговые Преобразования

```

alpha =
[-0.50000 -0.16667 -0.16667 -0.33333 -0.33333 -0.33333 0.00000 0.00000]
beta =
[4.50000 3.00000 -0.50000 -3.66667 -2.66667 -1.33333 2.00000 -2.00000]
x =
[3.00000 3.00000 0.00000 -3.00000 -2.00000 -2.00000 2.00000 -2.00000]

```

Листинг 3: Вывод

Как мы видим, мы получили матрицы α , β и x для данной матрицы, используя метод прогонки.

$$\vec{\alpha} = \begin{pmatrix} -0.5 \\ -0.16667 \\ -0.16667 \\ -0.33333 \\ -0.33333 \\ -0.33333 \\ 0 \\ 0 \end{pmatrix}, \vec{\beta} = \begin{pmatrix} 4.5 \\ 3 \\ -0.5 \\ -3.66667 \\ -2.66667 \\ -1.33333 \\ 2 \\ -2 \end{pmatrix}, \vec{x} = \begin{pmatrix} 3 \\ 3 \\ 0 \\ -3 \\ -2 \\ -2 \\ 2 \\ -2 \end{pmatrix}$$

И мы проверяем результат, умножая матрицу x на данную матрицу A .

```

testing = np.dot(A,x)
print_matrix(testing)

```

Листинг 4: Умножение матрицу x на данную матрицу A

```

[9.00000 27.00000 9.00000 -135.00000 -59.00000 -20.00000 8.00000 -2.00000]

```

Листинг 5: Результат умножения

Этот результат равен нашему $\vec{b}$.

Сейчас находим оценку невязки R используя норму $\|\cdot\|_{\infty}$.

```

residual = np.linalg.norm(A.dot(x) - d, ord = np.inf)
print("Residual ||.||_")
print(residual)

```

Листинг 6: Код, который находит норму $\|\cdot\|_{\infty}$.

```
Residual ||.||_∞  
0.0
```

Листинг 7: Оценка невязки по норме $\|\cdot\|_\infty$